

Functional Requirements-

1. User Registration & Authentication

- Users can sign up, log in, log out, and reset passwords securely
- Supports email/phone-based authentication

Impacts: Backend + Frontend

- Frontend: Forms, validation, UI flows
- Backend: User database, authentication logic, security

2. User Profile Management

- Users can create and edit profiles (name, bio, profile picture, etc.)
- View other users' profiles

Impacts: Both

- Frontend: Profile UI, edit screens
- Backend: Data storage, retrieval APIs

3. Content Creation (Posts: Text, Image, Video)

- Users can create, edit, and delete posts
- Support for media uploads (images/videos)

Impacts: Both

- Frontend: Post composer, media upload UI
- Backend: Media storage, processing, database handling

4. Feed & Content Discovery

- Users see posts from accounts they follow in a feed
- Basic chronological or relevance-based sorting

Impacts: Both

- Frontend: Feed UI, infinite scroll
- Backend: Feed generation logic, ranking, caching

5. Follow/Unfollow System

- Users can follow/unfollow others
- Maintain follower/following relationships

Impacts: Backend + Frontend

- Frontend: Buttons, follower counts
- Backend: Relationship management, graph storage

6. Direct Messaging (1:1 Chat)

- Users can send and receive private messages
- Real-time or near real-time delivery

Impacts: Both

- Frontend: Chat interface
- Backend: Messaging service, sockets/queues, storage

7. Community Groups

- Users can join/create groups
- Post and interact within groups

Impacts: Both

- Frontend: Group UI, navigation
- Backend: Group membership, permissions, content handling

Non-Functional Requirements-

1. Performance (Latency & Responsiveness)

- Feed load time ≤ 2 seconds under normal conditions
- API response time ≤ 300 ms for core actions (post, like, follow)
- Media upload should be optimized with compression and CDN delivery

Why it matters: Social apps are highly engagement-driven; slow UX leads to churn

2. Scalability

- System should handle horizontal scaling (microservices / cloud-native infra)

- Able to support spikes (e.g., viral posts, trending topics)
- Use load balancing and auto-scaling mechanisms

Why it matters: Social platforms grow unpredictably; infra must scale without downtime

3. Data Privacy & GDPR Compliance

- Must comply with General Data Protection Regulation (GDPR)
- Features required:
 - User consent management (cookies, tracking)
 - Right to access, rectify, and delete data (“Right to be Forgotten”)
 - Clear privacy policy and data usage transparency

Why it matters: Mandatory for operating in the EU; non-compliance leads to heavy penalties

4. Data Residency & Sovereignty

- All EU user data must be stored and processed within EU-based data centers
- Ensure compliance with cross-border data transfer restrictions

Why it matters: EU regulations require strict control over where user data resides

5. Security

- End-to-end encryption for direct messages
- Secure authentication (OAuth, 2FA support)
- Protection against:
 - XSS, CSRF, SQL injection
 - DDoS attacks
- Regular security audits and penetration testing

Why it matters: Social platforms are high-risk targets for breaches and abuse

6. Availability & Reliability

- Target uptime $\geq 99.9\%$
- Graceful degradation under failure (e.g., feed loads even if messaging fails)
- Backup and disaster recovery mechanisms in place

Why it matters: Users expect always-on access; downtime directly impacts trust

7. Content Moderation & Compliance (EU Regulations)

- Ability to detect and remove illegal/harmful content
- Compliance with Digital Services Act (DSA)
- Provide reporting mechanisms and transparency on moderation actions

Why it matters: EU mandates platform accountability for user-generated content

Data Model Design

1. User Profile Module

Entity: User

Key Attributes:

- user_id (PK)
- username (unique)
- email (unique)
- password_hash
- full_name
- bio
- profile_picture_url
- date_of_birth
- created_at
- updated_at
- account_status (active/suspended/deleted)

2.Entity: UserSettings

Key Attributes:

- settings_id (PK)
- user_id (FK → User)
- privacy_level (public/private/friends)
- notification_preferences (JSON)
- language
- theme

3.Entity: Follow

Key Attributes:

- follow_id (PK)
- follower_id (FK → User)
- following_id (FK → User)
- created_at

Relationships (User Module)

- One **User** → **UserSettings** (1:1)
- One **User** → **Many Followers** (1:N via Follow)
- One **User** → **Many Following** (1:N via Follow)

This forms a **self-referencing many-to-many relationship** via the Follow table.

2. Content Posting Module

Entity: Post

Key Attributes:

- post_id (PK)
- user_id (FK → User)
- content_text
- visibility (public/private/group)
- created_at
- updated_at
- is_deleted (boolean)

Entity: Media

Key Attributes:

- media_id (PK)
- post_id (FK → Post)
- media_type (image/video)
- media_url
- thumbnail_url
- size
- duration (for video)

Entity: Like

Key Attributes:

- like_id (PK)
- user_id (FK → User)
- post_id (FK → Post)
- created_at

Entity: Comment

Key Attributes:

- comment_id (PK)
- post_id (FK → Post)
- user_id (FK → User)
- parent_comment_id (nullable, for threaded comments)
- content_text
- created_at

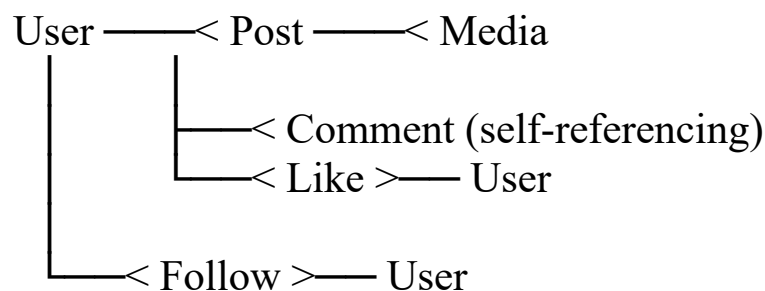
Relationships (Content Module)

- One **User** → **Many Posts** (1:N)
- One **Post** → **Many Media** (1:N)
- One **User** ↔ **Post (Likes)** (M:N via Like)
- One **User** → **Many Comments** (1:N)
- One **Post** → **Many Comments** (1:N)
- Comments support **self-referencing hierarchy**

Cross-Module Relationships

- **User** → **Post** (author relationship)
- **User** → **Like** → **Post** (engagement relationship)
- **User** → **Comment** → **Post** (interaction relationship)

ER Flow-



Design Considerations:

- Store **minimal personal data** → aligns with GDPR
- Use **soft deletes (is_deleted)** instead of hard deletes
- Keep **media storage separate** (S3/CDN)
- Use **indexed fields** on:
 - user_id
 - post_id
 - created_at

API Design

1. User Registration

- **Method:** POST
- **Endpoint:** /api/v1/auth/register

Input:

```
{
  "username": "neel123",
  "email": "neel@email.com",
  "password": "hashed_password"
}
```

Output:

```
{
  "user_id": "u123",
  "message": "User registered successfully"
}
```

Supports: User Registration & Authentication

2. User Login

- **Method:** POST
- **Endpoint:** /api/v1/auth/login

Input:

```
{
  "email": "neel@email.com",
  "password": "password"
}
```

Output:

```
{
```

```
"access_token": "jwt_token",  
"refresh_token": "refresh_token"  
}
```

Supports: Authentication

3. Get User Profile

- **Method:** GET
- **Endpoint:** /api/v1/users/{user_id}

Input:

- Path param: user_id

Output:

```
{  
  "user_id": "u123",  
  "username": "neel123",  
  "bio": "MBA | Analytics",  
  "profile_picture_url": "url",  
  "followers_count": 120,  
  "following_count": 80  
}
```

Supports: Profile Management

4. Get User Profile

- **Method:** PUT
- **Endpoint:** /api/v1/users/{user_id}

Input:

```
{  
  "bio": "Updated bio",  
  "profile_picture_url": "new_url"  
}
```

Output:

```
{  
  "message": "Profile updated successfully"  
}
```

Supports: Profile Management

5. Create Post

- **Method:** POST

- **Endpoint:** `/api/v1/posts`

Input:

```
{
  "content_text": "Hello ConnectEU!",
  "media_urls": ["img1.jpg"],
  "visibility": "public"
}
```

Output:

```
{
  "post_id": "p456",
  "message": "Post created"
}
```

Supports: Content Creation

6. Get Feed

- **Method:** GET
- **Endpoint:** `/api/v1/feed`

Input:

- Query params: `page, limit`

Output:

```
{
  "posts": [
    {
      "post_id": "p456",
      "user": {
        "user_id": "u123",
        "username": "neel123"
      },
      "content_text": "Hello!",
      "media_urls": [],
      "likes_count": 10,
      "comments_count": 2,
      "created_at": "timestamp"
    }
  ]
}
```

Supports: Feed & Content Discovery

7. Follow/Unfollow User

- **Method:** POST / DELETE
- **Endpoint:** /api/v1/users/{user_id}/follow

Input:

- Path param: user_id

Output:

```
{
  "message": "Followed successfully"
}
```

Supports: Follow System

8.Like/Unlike Post

- **Method:** POST / DELETE
- **Endpoint:** /api/v1/posts/{post_id}/like

Input:

- Path param: post_id

Output:

```
{
  "message": "Post liked"
}
```

Supports: Engagement (Content Interaction)

9.Send Direct Message

- **Method:** POST
- **Endpoint:** /api/v1/messages

Input:

```
{
  "receiver_id": "u456",
  "message_text": "Hello!"
}
```

Output:

```
{
  "message_id": "m789",
  "status": "sent"
}
```

Supports: Direct Messaging

API	Method	Requirement Supported
Register	POST	Authentication
Login	POST	Authentication
Get Profile	GET	Profile
Update Profile	PUT	Profile
Create Post	POST	Content Creation
Get Feed	GET	Feed
Follow/Unfollow	POST/DELETE	Social Graph
Like/Unlike	POST/DELETE	Engagement
Send Message	POST	Messaging

Success Metrics

1. Activation Rate

Definition: % of users who complete key first actions
(e.g., sign up → follow ≥ 5 users → view feed)

Why it matters:

Shows whether new users understand and get value quickly

Target (example): 60–70%

2. DAU / MAU (Engagement Health)

Definition: Daily Active Users $\div$ Monthly Active Users

Why it matters:

Indicates how “sticky” the platform is

Benchmark:

- ~20% = average
- ~30%+ = strong

3. Content Creation Rate

Definition: % of active users who create at least 1 post

Why it matters:

Content is the backbone of a social platform

Target: 20–30%

4. Feed Engagement Rate

Definition:

$(\text{Likes} + \text{Comments} + \text{Shares}) \div \text{Total posts viewed}$

Why it matters:

Measures whether your feed is relevant and engaging

5. Day 7 Retention

Definition: % of users returning 7 days after signup

Why it matters:

Tells you if users find long-term value

Benchmark: 25–40% (good for early-stage social apps)

6. Social Graph Density

Definition: Avg. number of follows per user

Why it matters:

A strong network = better feed quality + retention

Target: ≥ 10 follows per user early on

7. Time to First Value (TTFV)

Definition: Time taken for a new user to:

- see relevant content OR
- receive first like/follow

Why it matters:

Critical for onboarding success

Goal: < 5 minutes

8. Trust & Compliance Metric

Definition: % of content reports resolved within SLA

Why it matters:

Aligns with EU regulations like
Digital Services Act